

**操作系统**

**第3章 外设与文件**

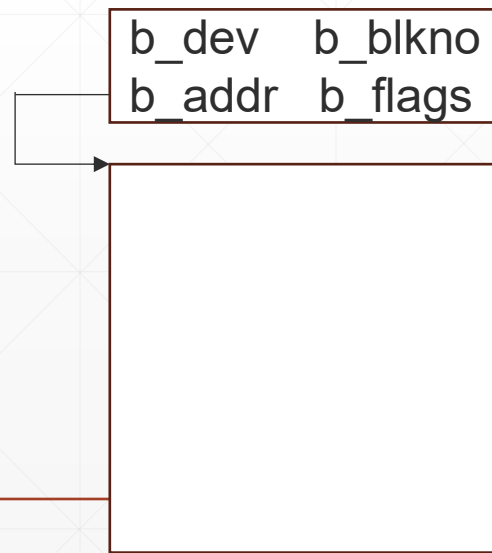
## **(1) 外设管理 (下)**

---

同济大学计算机系

# 缓存池的使用

- 1个缓存块可以装1个磁盘数据块。15个缓存块，最多装15个不同的磁盘数据块。
- 缓存命中的数据块供所有进程共享。为了保证数据不出错，要互斥。
  - 用前上锁。如果其它进程在用，入睡等待
  - 读写缓存块中的磁盘数据块复本 。 。 。 缓存块中的数据 $\leftrightarrow$ 进程图像（用户空间）
  - 用完释放。如果其它进程等待使用，WakeUpAll(&buf)
- 缓存不命中
  - 为磁盘数据块分配一个缓存块，上锁
  - 设置buf（IO请求块），读入磁盘数据块
  - 使用缓存块中的数据块复本
  - 解锁（同上）



# 缓存块的状态

- 空白。系统初始化时，15个空白缓存块，没有磁盘数据。可以分配给任意新数据块  
b\_dev b\_blkno 为 -1 (任意空标识)
- 保存有一个磁盘数据块复本  
b\_dev b\_blkno 非空
  - 磁盘数据块最新版本，数据块可用 (B\_DONE为1)
  - 不是，数据块不可用，读IO尚未完成 (B\_DONE为0)
  - 上锁 ( B\_BUSY为1，有进程正在用)
  - 自由 ( B\_BUSY为0，没人在用)

# 上锁

## bp指向的Buf, 管理分配给dev,blkno的缓存块

缓存池搜索  
目标数据块

遍历m\_buf数组, 搜索 b\_dev==dev, b\_blkno==blkno 的缓存块

```
loop: .....  
if(bp->b_flags & Buf::B_BUSY)  
{  
    bp->b_flags |= Buf::B_WANTED;  
    u.u_procp->Sleep((unsigned long)bp, ProcessManager::PRIBIO);  
    goto loop;  
}  
bp->b_flags |= Buf::B_BUSY, return bp;    // B_DONE是1
```

缓存  
命中

缓存不  
命中

为dev,blkno分配一个缓存块（最近最久未使用），bp指向其控制块  
b\_dev = dev, b\_blkno = blkno  
b\_flags = B\_BUSY  
return bp // B\_DONE是0



# 解锁

## bp指向的Buf

```
if(bp->b_flags & Buf::B_WANTED)
{
    procMgr.WakeUpAll((unsigned long)bp);
}
bp->b_flags &= ~Buf::B_BUSY
```

## 5、Unix V6++ 外设管理

- ID是外设的内部标识符，唯一。
  - 主设备号major，标识外设类型。相同major的不同设备，共用驱动程序和中断处理程序。
  - 从设备号minor，标识外设个体，是同类型的第几个设备。

设备号    short   b\_dev;



- Unix V6，块设备 和 字符设备独立管理。

外设管理模块的  
核心数据结构

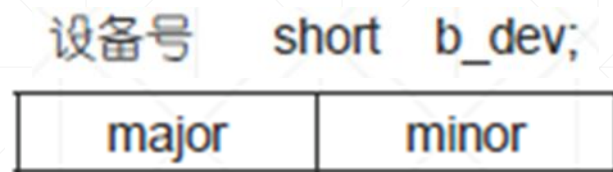
```
class DeviceManager
{
    .....
    int nblkdev;           // 块设备的类型数
    BlockDevice *bdevsw[MAX_DEVICE_NUM]; // 块设备开关表，每类块设备一项

    int nchrdev;           // 字符设备的类型数
    CharDevice *cdevsw[MAX_DEVICE_NUM];  // 字符设备开关表，每类字符设备一项
}
```



```
class DeviceManager
{
    .....
    int nblkdev;          // 块设备的类型数
    BlockDevice *bdevsw[MAX_DEVICE_NUM]; // 块设备开关表, 每类块设备一项

    int nchrdev;          // 字符设备的类型数
    CharDevice *cdevsw[MAX_DEVICE_NUM]; // 字符设备开关表, 每类字符设备一项
}
```



```
class BlockDevice
{
public:
    BlockDevice();
    BlockDevice(Devtab** tab);
    virtual ~BlockDevice();
    virtual int Open(short dev, int mode); // 盘的打开函数
    virtual int Close(short dev, int mode); // 盘的关闭函数
    virtual int Strategy(Buf* bp);          // 提交IO请求
    virtual void Start();                   // 启动IO操作

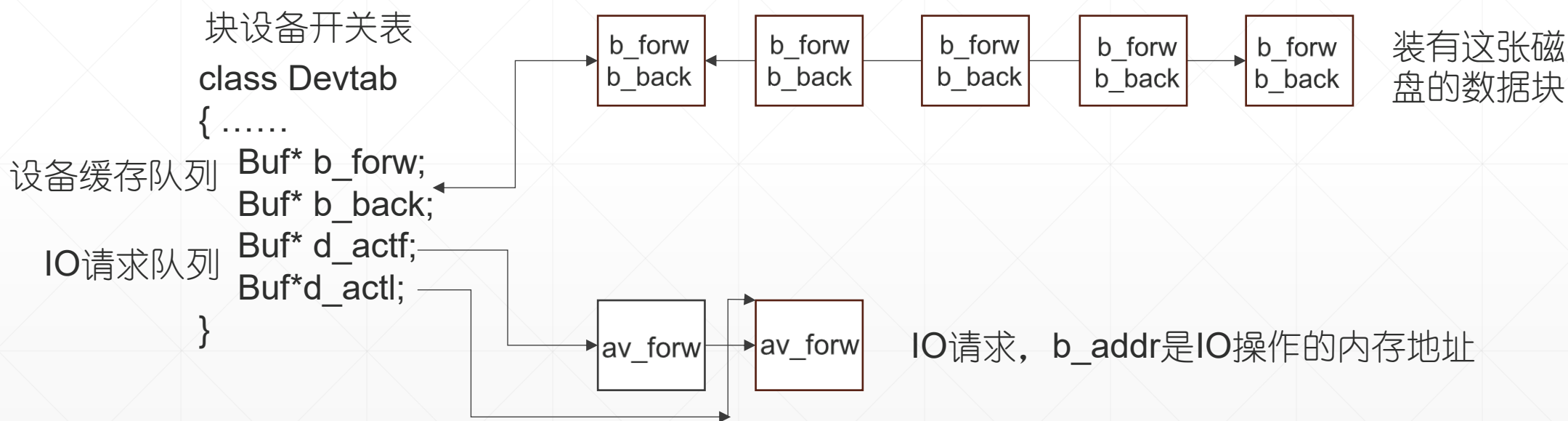
public:
    Devtab* d_tab[N]; // 这种盘配置了N个
};
```

```
class Devtab // 每张盘1个块设备表。块设备表
{ // 是管理磁盘的控制块, 记录磁盘的使用现状
public:
    Devtab();
    ~Devtab();

public:
    int d_active; // 忙, 正在执行IO操作
    int d_errcnt; // 队首IO操作的失败次数
    Buf* b_forw; // 设备请求队列
    Buf* b_back; // 每张磁盘一根
    Buf* d_actf; // IO请求队列
    Buf* d_actl; // 每张磁盘一根
};
```

## 6、Unix V6++ 磁盘高速缓存管理

每张磁盘，一根设备缓存队列 和 一根IO请求队列





# 自由缓存队列

系统维护一根自由缓存队列，除初始化阶段，缓存块中的数据可供其它进程重用。

```
class BufferManager // 管理磁盘高速缓存的对象
```

```
{ .....
```

```
private:
```

```
    Buf bFreeList; // 自由缓存队列的队首
```

```
    .....
```

```
}
```

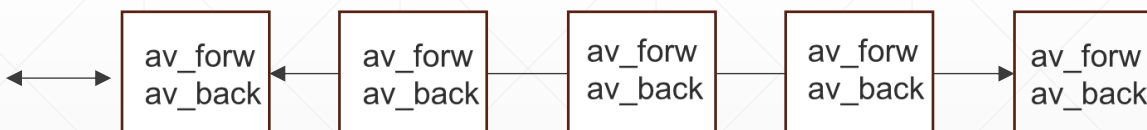
```
    class Buf
```

```
    {
```

```
        Buf*av_forw;
```

```
        Buf*av_back;
```

```
    }
```



自由缓存

## 缓存块分配策略

缓存不命中时，淘汰最近最久未访问（Least Recently Used, LRU）磁盘数据块，用它的缓存块装新数据块

# 自由缓存是LRU队列

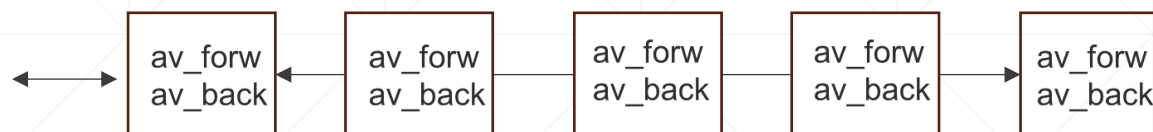
```
class Buf
```

```
{
```

```
    Buf*av_forw;
```

```
    Buf*av_back;
```

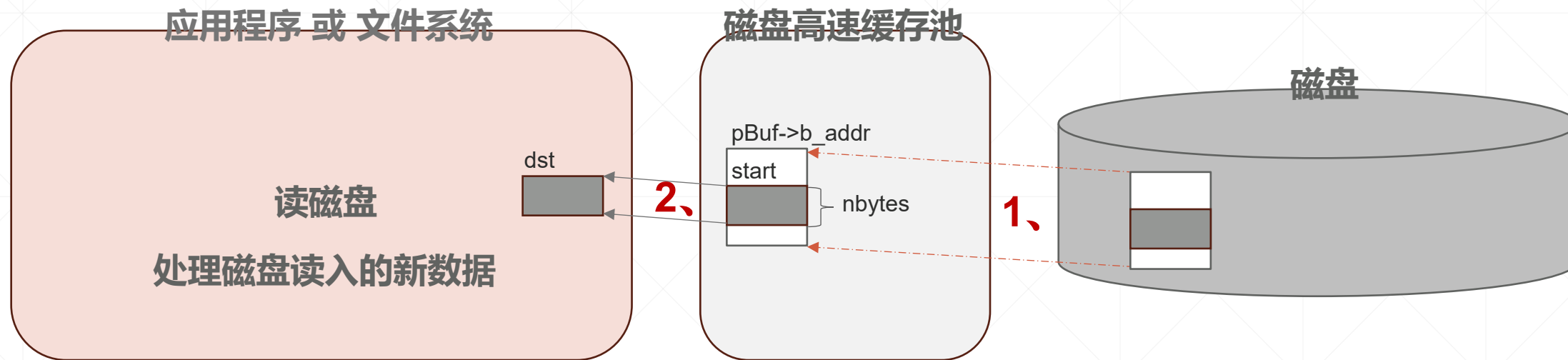
```
}
```



缓存块使用前上锁，从自由缓存队列中摘除。  
使用完毕解锁，送自由缓存队列尾部。

为新数据块分配队首缓存（上锁、移除），时间复杂度 $O(1)$ 。

## 7 同步读 (Unix 基本读操作)



`pBuf = bufMgr.Bread(dev, bn);` //1、将磁盘数据块<dev,bn>读入缓存池，pBuf缓存块，上锁

// 2、读取缓存块中的数据，送进程用户空间（应用程序定义的变量）

`unsigned char* start = pBuf->b_addr + offset;` // start，数据在缓存块pBuf中的首地址。

`Utility::IOMove(start, dst, nbytes);` // dst，数据在用户空间的首地址。nbytes，数据量。

`bufMgr.Brelse(pBuf);` // 3、缓存块pBuf使用完毕，解锁。

```
void Utility::IOMove(unsigned char* from,
                    unsigned char* to, int count)
{
    while(count-->0)
        *to++ = *from++;
    return;
}
```

后续：这个数据块会一直留在缓存池中，等待其它进程重用；直至成为最近最久未访问

(Least Recently Used, LRU) 数据块，被缓存池淘汰。



Buf\* BufferManager::Bread(short dev, int blkno) // 读磁盘dev、blkno#扇区

```
{
    Buf* bp;

    bp = this->GetBlk(dev, blkno); // 1、锁住缓存池中分配给blkno的缓存块

    if(bp->b_flags & Buf::B_DONE) // 2、blkno缓存命中，返回
    {
        return bp;
    }

    bp->b_flags |= Buf::B_READ; // 3、否则，构造读IO请求块
    bp->b_wcount = BufferManager::BUFFER_SIZE;
    this->m_DeviceManager->GetBlockDevice(Utility::GetMajor(dev)).Strategy(bp); // 4、将IO请求块送IO
    this->IOWait(bp); // 同步（入睡）等待IO操作结束 请求队列末尾
    return bp; // 返回
}
```

返回装有磁盘数据块副本的缓存块（已上锁）

# bp = Getblk( dev, blkno )

锁住缓存池中分配给blkno的缓存块, bp

- 搜索缓存池，寻找分配给< dev, blkno >的缓存块（缓存控制块buf b\_dev==dev, b\_blkno==blkno）
- 找到（缓存命中， Buffer[i] ）
  - B\_BUSY为1？ 是：置B\_WANTED标识（ m\_Buf[i].b\_flags |= Buf::B\_WANTED ），睡眠等待持锁进程解锁（ B\_BUSY变0 ）。  
Sleep(& m\_Buf[i], -50)
  - 上锁（ m\_Buf[i].b\_flags |= Buf::B\_BUSY ），自由缓存队列出列。
  - 返回 m\_Buf[i]首地址。 // 分支1。进程持有Buffer[i]的使用权， B\_DONE为1、可用缓存块。
- 没找到（缓存不命中），为<dev, blkno>分配缓存块Buffer[j]。
  - 自由缓存队列第一个不脏的缓存块。遍历过程中，顺带将所有脏缓存块写回磁盘。
  - 重置Buffer[j]的属性（下页PPT，此处细节略）：  
m\_Buf[j].b\_dev=dev ; m\_Buf[j].b\_blkno = blkno;  
m\_Buf[j].b\_flags = Buf::B\_BUSY // 上锁 & 清除所有其余标识，尤其B\_DONE
  - 自由缓存队列出列。将缓存块从原设备缓存队列移除，插入新设备缓存队列队首。
  - 返回 m\_Buf[j]首地址。 // 分支2。进程持有Buffer[j]的使用权， B\_DONE为0、空缓存块。

若自由缓存队列空，入睡等待自由缓存  
Sleep(&bFreeList, -50)



# IOWait(bp)

同步（入睡）等待I/O操作结束

```
void BufferManager::IOWait(Buf* bp)
{
    User& u = Kernel::Instance().GetUser();

    X86Assembly::CLI();
    while( (bp->b_flags & Buf::B_DONE) == 0 )
    {
        u.u_procp->Sleep((unsigned long)bp, ProcessManager::PRIBIO);
    }
    X86Assembly::STI();

    this->GetError(bp);
    return;
}
```

# 磁盘中断处理程序

## void ATADriver::ATAHandler (struct pt\_regs \*reg, struct pt\_context \*context)

- 1、摘下IO请求队列队首Buf
- 2、IODone(bp);
- 3、启动下一个I/O操作

```
void BufferManager::IODone(Buf* bp)
{
    bp->b_flags |= Buf::B_DONE;           //标记IO完成
    if(bp->b_flags & Buf::B_ASYNC)
        this->Brelse(bp);
    else {
        bp->b_flags &= (~Buf::B_WANTED);
        WakeUpAll((unsigned long)bp);
    }
    return;
}
```

IO请求队列不空？  
yes，向磁盘提交新队首IO请求。

# UNIX V6++的硬盘驱动（中断处理程序）

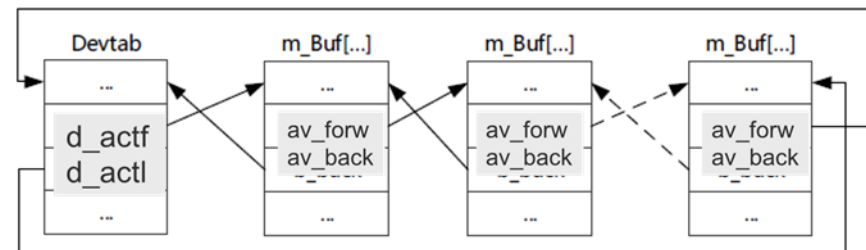


```
void ATADriver::ATAHandler(struct pt_regs *reg, struct pt_context *context)
{
    .....
    short major = Utility::GetMajor(DeviceManager::ROOTDEV);
    BlockDevice& bdev = Kernel::Instance().GetDeviceManager().GetBlockDevice(major);
    atab = bdev.d_tab;
    .....
    bp = atab->d_actf; /* 定位I/O请求队列的队首缓存 */
    atab->d_active = 0; /* 标设备空闲 */

    if( ATADriver::IsError() || DMA::IsError() ) // 检查错误
    {
        if(++atab->d_errcnt <= 10)
        {
            bdev.Start(); //偶然出错，比如磁头上有静电或浮灰。Retry
            return;
        }
        bp->b_flags |= Buf::B_ERROR; // 扇区真的坏了
    }
}
```

```
class Devtab // 块设备表
{
public:
    int d_active; // 磁盘忙，IO请求队列非空
    int d_errcnt; // 队首IO操作，出错次数
    Buf* b_forw; // 设备缓存队列队首
    Buf* b_back;
    Buf* d_actf; // IO请求队列队首
    Buf* d_actl;
};
```

IO请求队列







```
atab->d_errcnt = 0; /* 错误计数器归零 */
atab->d_actf = bp->av_forw; /* I/O请求队列, 摘队首buf */
Kernel::Instance().GetBufferManager().IODone(bp); /* 置1 B_DONE。唤醒等待IO的进程 */
bdev.Start(); /* 启动I/O请求队列中下一个I/O请求 */

/* 对主、从8259A中断控制芯片分别发送EOI命令。 */
IOPort::OutByte(Chip8259A::MASTER_IO_PORT_1, Chip8259A::EOI);
IOPort::OutByte(Chip8259A::SLAVE_IO_PORT_1, Chip8259A::EOI);
return;
}

void BufferManager::IODone(Buf* bp)
{
    bp->b_flags |= Buf::B_DONE; // 置IO完成标记
    if(bp->b_flags & Buf::B_ASYNC)
        .....
    else
    {
        // 对同步块
        bp->b_flags &= (~Buf::B_WANTED); // 清等待标记
        .....WakeUpAll((unsigned long)bp);
    }
    return;
}

void BufferManager::IOWait(Buf* bp)
{
    User& u = Kernel::Instance().GetUser();

    X86Assembly::CLI();
    while( (bp->b_flags & Buf::B_DONE) == 0 )
        u.u_procp->Sleep((unsigned long)bp, .....PRIBIO);
    X86Assembly::STI();

    this->GetError(bp);
    return;
}
```



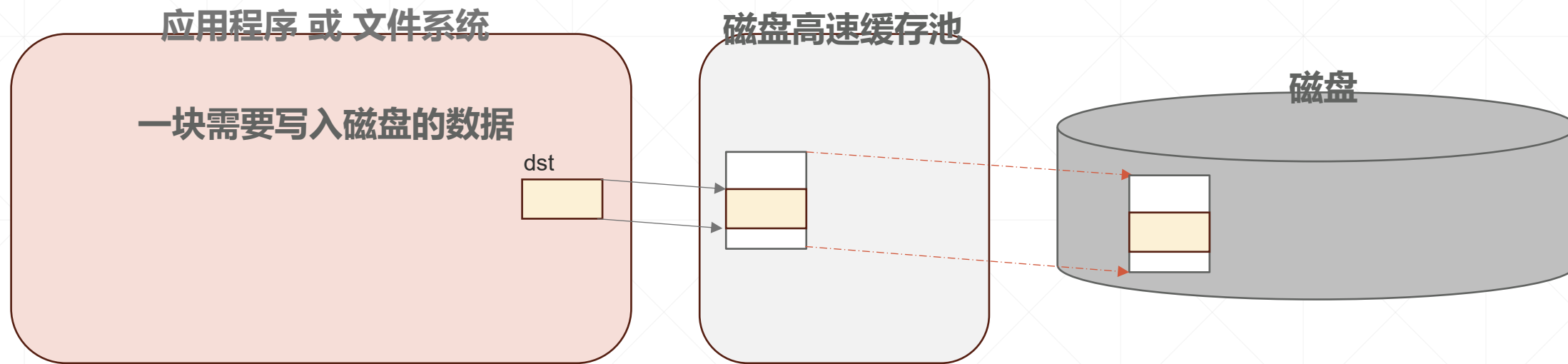
# Brelse(Buf\* bp)

释放缓存块bp的使用权

```
void BufferManager::Brelse(Buf* bp)
{
    .....
    if(bp->b_flags & Buf::B_WANTED)
    {
        procMgr.WakeUpAll((unsigned long)bp);    // 唤醒等待使用数据块的进程
    }
    if(this->bFreeList.b_flags & Buf::B_WANTED)
    {
        this->bFreeList.b_flags &= (~Buf::B_WANTED);
        procMgr.WakeUpAll((unsigned long)&this->bFreeList);    // 唤醒等待自由缓存的进程
    }

    .....
    X86Assembly::CLI();    // 缓存块放自由缓存队列队尾
    bp->b_flags &= ~(Buf::B_WANTED | Buf::B_BUSY | Buf::B_ASYNC);
    (this->bFreeList.av_back)->av_forw = bp;
    bp->av_back = this->bFreeList.av_back;
    bp->av_forw = &(this->bFreeList);
    this->bFreeList.av_back = bp;
    X86Assembly::STI();
    return;
}
```

## 8 异步、延迟写 (Unix 基本写操作)



数据写入缓存池后，写操作完毕。

需要注意的技术细节：

- (1) 如果写入磁盘数据块的数据量不足512字节。**先读后写**，保护磁盘数据块中新数据覆盖不到的旧数据。
- (2) 数据块写满（写操作覆盖最后一个字节）时，异步写回磁盘。
  - 写满，指写操作覆盖最后一个字节
  - 若写不满，打脏标识、解锁缓存块，写操作完毕。

缓存块未写满，写盘操作延迟。

这是操作系统普遍使用的**延迟写技术**。延迟写，产生脏缓存（延迟写缓存）。脏缓存何时写回磁盘？写满、缓存块分配给其它磁盘数据块、关机或卸载磁盘。

## 向数据块<dev, blkno>写入count字节，块内起始偏移量offset

- `count == 512 ?`
  - `Y, bp = Getblk( dev, blkno )` // 缓存块写满，无需先读。
  - `N, bp = Bread( dev, blkno )` // 先读
- 数据写进缓存池
  - `unsigned char* start = bp->b_addr + offset;` // start, 缓存块写入位置的首地址。
  - `Utility::IOMove(dst, start, count);` // dst, 用户区中数据块的首地址。
- `count + offset == 512 ?`
  - `Y, Bawrite(bp)` // 写满的缓存块异步写回磁盘（放IO队列尾部，不睡）。
  - `N, Bdwrite(bp)` // 没写满的缓存块，直接释放，放自由缓存队列尾部。

后台：异步写完成后，磁盘中断处理程序将缓存块放自由缓存队列尾部。

返回 // 写操作完成后，缓存块中的数据块是磁盘数据块的最新复本。

# 写操作，源代码 1

```
void BufferManager::Bdwrite(Buf *bp) // 延迟写。缓存内容不写盘
{
    /* 置上B_DONE允许其它进程使用该磁盘块内容 */
    bp->b_flags |= (Buf::B_DELWRI | Buf::B_DONE);
    this->Brelse(bp);
    return;
}

void BufferManager::Bawrite(Buf *bp) // 异步写。缓存内容写盘
{
    /* 置异步写标识，IO完成后由磁盘中断处理程序释放缓存 */
    bp->b_flags |= Buf::B_ASYNC;
    this->Bwrite(bp); // 缓存块写盘
    return;
}
```



## 写操作, 源代码 2

```
void BufferManager::Bwrite(Buf *bp)
{
    unsigned int flags;

    flags = bp->b_flags;
    bp->b_flags &= ~(Buf::B_READ | Buf::B_DONE | Buf::B_ERROR | Buf::B_DELWRI);
    bp->b_wcount = BufferManager::BUFFER_SIZE;
    this->m_DeviceManager->GetBlockDevice(Utility::GetMajor(bp->b_dev)).Strategy(bp);

    if( (flags & Buf::B_ASYNC) == 0 )
    {
        /* 同步写, 需要等待I/O操作结束 */
        this->IOWait(bp);
        this->Brelse(bp);
    }
    else if( (flags & Buf::B_DELWRI) == 0)
    {
        this->GetError(bp);
    }
    return;
}
```

## 9 Unix V6++的硬盘驱动

- IO请求队列

```
class Devtab // 块设备表
{
public:
    int d_active; // 磁盘忙, IO请求队列非空
    int d_errcnt; // 队首IO操作, 出错次数
    Buf* b_forw; // 设备缓存队列队首
    Buf* b_back;
    Buf* d_actf; // IO请求队列队首
    Buf* d_actl;
};
```

- 设备驱动程序 和 中断处理程序



# ATA硬盘驱动程序

缓存层或进程管理模块发出IO请求。bp是一个 m\_buf[i] 或 Swbuf。

```
int ATABlockDevice::Strategy(Buf* bp)
{
    .....
    bp->av_forw = NULL;

    X86Assembly::CLI();

    IO控制块插入IO请求队列尾部;
    if(this->d_tab->d_active == 0)
    { /* 原先磁盘空闲, 立即发IO命令 */
        this->Start();
    }

    X86Assembly::STI();

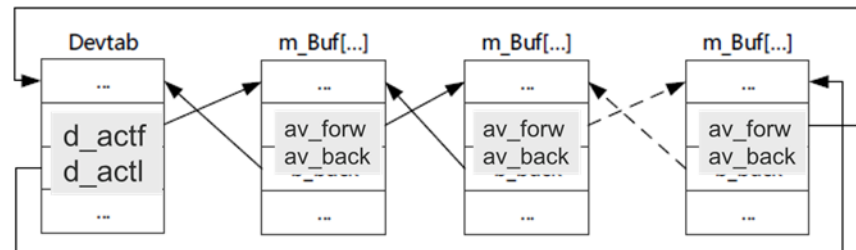
    return 0;          /* GCC likes it ! */
}
```

```
void ATABlockDevice::Start() // 启动IO请求队列队首IO操作
{
    Buf* bp;

    if( (bp = this->d_tab->d_actf) == 0 )
        return; /*IO请求队列空, 返回 */

    /* IO请求队列不空, 设置控制器忙标志 */
    this->d_tab->d_active++;
    /* 编程硬件, 启动IO操作 */
    ATADriver::DevStart(bp);
}
```

IO请求队列





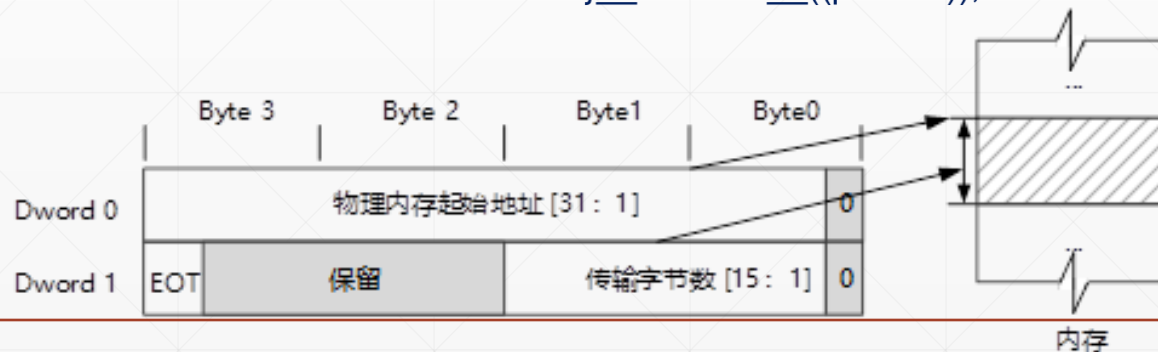
# void ATADriver::DevStart(struct Buf\* bp), 发磁盘IO命令

## 一、编程DMA控制器： 设置内存物理地址，数据传输量

写PRD表。PRD表，最多10项。每一项是一个PRD（Physical Region Descriptor，物理内存区描述符），管理DMA操作访问的一块连续内存区域。

```
class PRDTable
{
public:
    static const int NSIZE = 10;
    .....
private:
    PhysicalRegionDescriptor m_Descriptors[NSIZE];
};
```

```
class PhysicalRegionDescriptor /// 每个元素，一块连续内存区域
{
private:
    unsigned long m_BaseAddressZeroBit : 1; // 0
    unsigned long m_MemoryRegionPhysicalBaseAddress : 31; // 首地址
    unsigned short m_ByteCountZeroBit : 1; // 0
    unsigned short m_ByteCount : 15; // 长度
    .....
    unsigned short m_EOT : 1; // End of Table位。1表示当前prd是最后一项
}__attribute__((packed));
```





```
class Buf
{ // 管理缓存块：使用状态 和 IO参数。
public:
    unsigned int b_flags; // 位图，登记缓存使用状态
    int padding; // 技术细节，看代码

    Buf*    b_forw; // 队列指针，用来管理缓存
    Buf*    b_back;
    Buf*    av_forw;
    Buf*    av_back;

    unsigned char* b_addr; // Buffer[i] 首地址
    short    b_dev; // 磁盘号
    int      b_blkno; // 扇区号
    int      b_wcount; // IO参数，传送字节数
    int      b_error; // I/O错误info
    int      b_resid; // I/O出错时尚未传送的剩余字节数
};
```

// 设置一个PRD

prd.SetBaseAddress((unsigned long)bp->b\_addr & ~0xC0000000); // 首地址（物理地址）

prd.SetByteCount(bp->b\_wcount); // 文件系统数据块 512字节；进程图像，任意长度

table.SetPhysicalRegionDescriptor(0, prd, true); // 写进PRD表，第0项

DMA::Reset(); /\* 复位DMA控制器 \*/



## 二、编程磁盘控制器： 设置需要IO的磁盘扇区号，扇区数。

```
/* IO扇区数 */
IOPort::OutByte(ATADriver::NSECTOR_PORT, bp->b_wcount / BufferManager::BUFFER_SIZE);

/* 起始扇区号 */
IOPort::OutByte(ATADriver::BLKNO_PORT_1, bp->b_blkno & 0xFF);
IOPort::OutByte(ATADriver::BLKNO_PORT_2, (bp->b_blkno >> 8) & 0xFF);
IOPort::OutByte(ATADriver::BLKNO_PORT_3, (bp->b_blkno >> 16) & 0xFF);

/* 设置ATA磁盘工作模式寄存器，以及LBA28中的24-27位 */
IOPort::OutByte(ATADriver::MODE_PORT,
    ATADriver::MODE_IDE | ATADriver::MODE_LBA28 | (minor << 4) | ((bp->b_blkno >> 24) & 0x0F) );
```



### 三、向磁盘控制器和DMA控制器发IO命令

```
/* 如果是读操作 */
if( (bp->b_flags & Buf::B_READ) == Buf::B_READ )
{
    /* 告诉磁盘控制器的读、写类型，启动I/O */
    IOPort::OutByte(ATADriver::CMD_PORT, ATADriver::HD_DMA_READ);

    /* 告诉DMA控制器的读、写类型，传入PRD Table的物理起始地址，启动一次DMA */
    DMA::Start(DMA::READ, table.GetPRDTableBaseAddress());
}
else /* 如果是写操作 */
{
    IOPort::OutByte(ATADriver::CMD_PORT, ATADriver::HD_DMA_WRITE);

    DMA::Start(DMA::WRITE, table.GetPRDTableBaseAddress());
}
return;
```

# 总结： Unix V6++ 外设管理模块 和 缓存管理模块

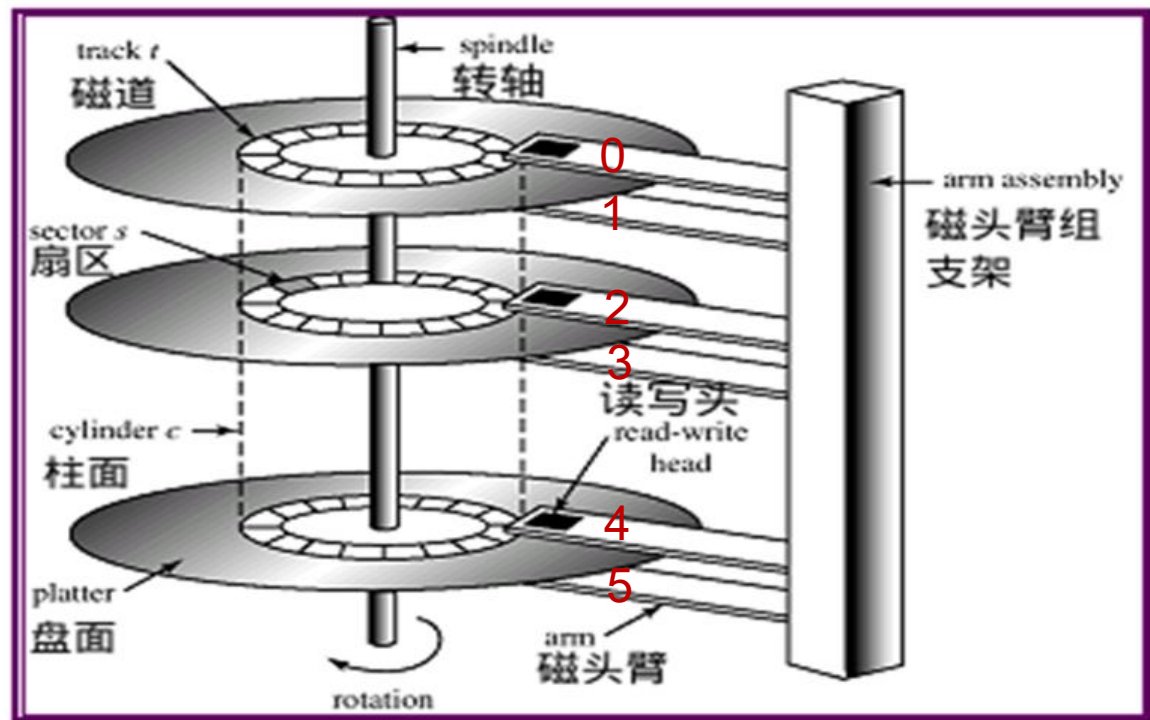




## 10、IO性能优化（考虑机械硬盘特性）

# 机械硬盘（等速圆周运转的设备）

- 机械硬盘，同心圆盘。
  - 每个面，布有用来存储信息的许多同心磁道。从外至内编号，0, 1, 2, 3... 磁道号。所有盘面，相同磁道号的磁道组成柱面。柱面号就是磁道号。
  - 每根磁道，许多等圆心角圆弧。每个圆弧1个扇区。0, 1, 2, 3... 扇区号。
  - 每张圆盘，上下2个盘面，每个盘面配有一个用来读写的磁头。磁头编号，0, 1, 2, 3... 磁头号。
- 磁盘，信息存储、访问单位为扇区，目前的技术标准是每个扇区存放512字节。扇区的物理地址：柱面号、磁头号、扇区号。序列化，得逻辑扇区号。

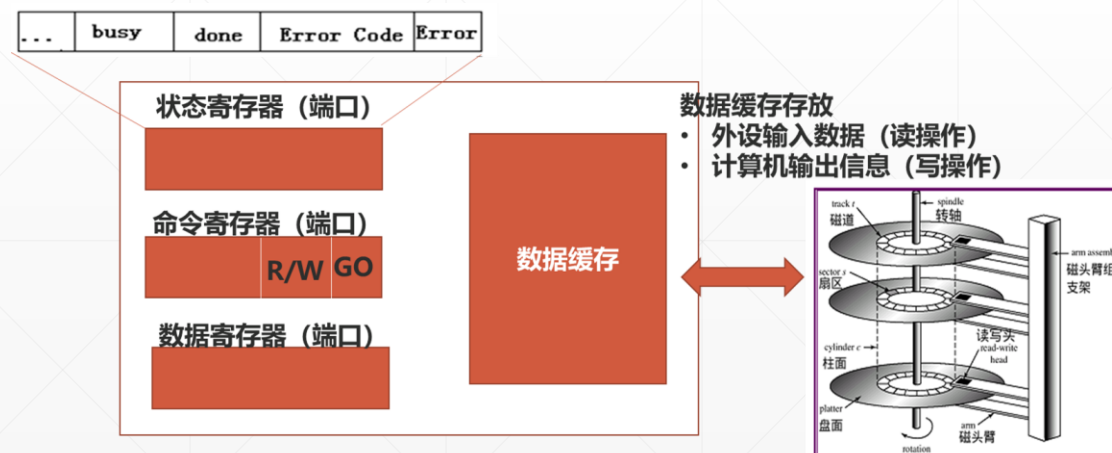


0#柱面, 0#磁头: 逻辑扇区号 0~499  
0#柱面, 1#磁头: 逻辑扇区号 500~999  
1#柱面, 0#磁头: 逻辑扇区号 1000~1499  
1#柱面, 1#磁头: .....



# 硬盘的访问耗时：从CPU写命令寄存器至数据块（512字节）进硬盘控制器数据缓存

- **Seek time**寻道时间：磁头从当前柱面移动到目标柱面的时间。平均寻道时间小于10ms。
- **Rotational delay**旋转延时：磁头到达目标柱面（磁道）后，等待欲访问扇区转到磁头之下。平均旋转延时为磁盘旋转一周所需时间 $T$ 的 $1/2$ 。
- **Data transfer**数据传输时间：磁头走过目标扇区，期间读扇区内容至硬盘数据缓存。若每个磁道包含 $N$ 个扇区。数据传输时间为： $T/N$ 。



# 优化寻道时间

- 磁臂调度算法重排IO请求，优化总寻道时间

例：磁盘IO请求分布在磁道：55, 58, 39, 18, 90, 160, 150, 38, 184;

当前磁道号：100。

常用的磁臂调度算法：

## 1、FIFS (FIFO) :

- 响应次序：55, 58, 39, 18, 90, 160, 150, 38, 184。
- 磁头移动轨迹：100, 55, 58, 39, 18, 90, 160, 150, 38, 184。
- 磁臂移动平均距离：55.3个磁道

sda0的块设备表

```
• 块设备表：登记外设的工作现状
struct Devtab {
    char    d-active;
    char    d-errcnt;
    struct buf *b-forw;
    struct buf *b-back;
    struct buf *d-actf;
    struct buf *d-actl;
};
```

0  
213  
读

0#缓存块

0  
500  
写

1#缓存块

例：磁盘IO请求分布在磁道：55, 58, 39, 18, 90, 160, 150, 38, 184;  
当前磁道号：100。

- **SSTF** (Shortest Service Time First) :优先响应离磁头当前位置近的请求

响应次序：90, 58, 55, 39, 38, 18, 150, 160, 184;

磁头移动轨迹：100, 90, 58, 55, 39, 38, 18, 150, 160, 184。

磁臂移动平均距离：27.5个磁道

- **SCAN** (电梯调度算法)：考虑磁头的当前位置和运动方向，依次服务途径的所有磁道上的服务请求。当磁头到达最高或最低磁道时，改变运动方向。

假设磁头目前正在向大号磁道运动

150, 160, 184, 90, 58, 55, 39, 38, 18

磁臂移动平均距离：27.8个磁道

例：磁盘IO请求分布在磁道：55, 58, 39, 18, 90, 160, 150, 38, 184;

当前磁道号：100。假设磁头目前正在向大号磁道运动。

- **C-SCAN**：与SCAN类似，磁头始终一个方向运动。

150, 160, 184, 18, 38, 39, 55, 58, 90;

磁臂移动平均距离：35.8个磁道

- 算法性能比较

- SSTF性能较好，但可能出现被饿死的IO请求。
- SCAN和C-SCAN理论上不会饿死IO请求，但是会出现“磁臂粘着”（Arm stickiness）现象。指：一个或几个进程对某个柱面密集访问，反复读写其中的磁道，导致磁臂停留此处不动，其它柱面的IO请求会长期得不到处理。
- 解决方法：设置多个磁盘IO请求集合

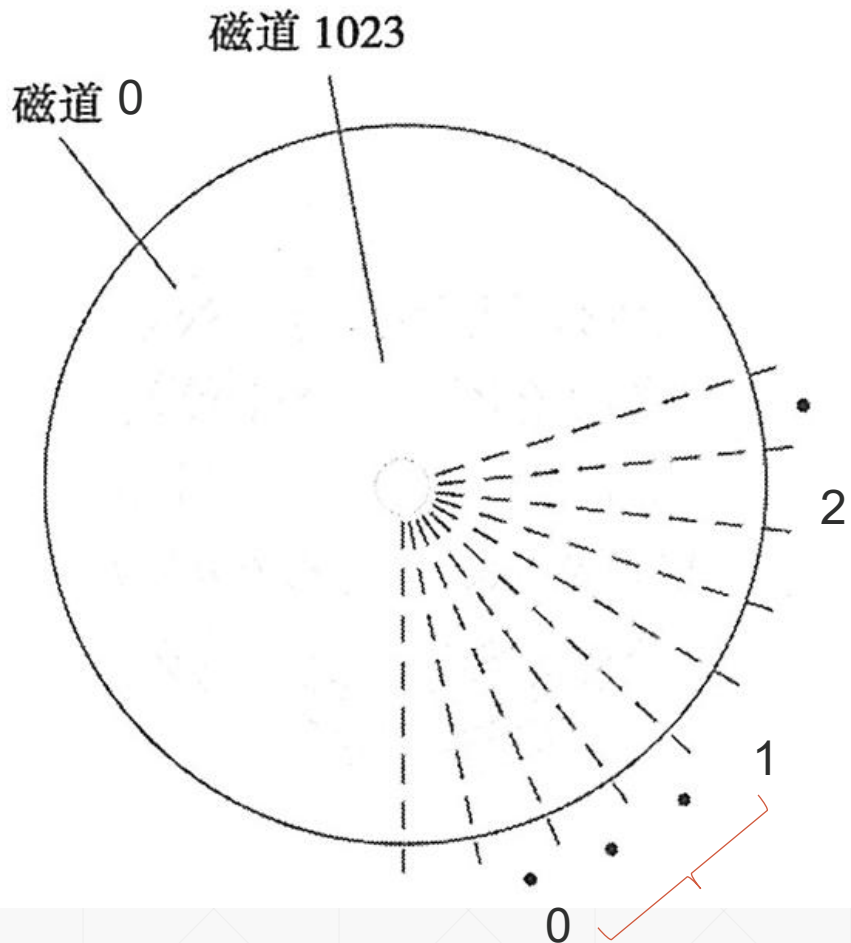
# N-Step-SCAN和FCSCAN

- N个IO请求队列。
- 磁盘服务其中的一个队列。服务期间，新IO请求放在第一个可供使用的新队列。
- 磁盘FIFS地服务所有队列。
- 当前服务队列空，服务下一个队列。
- 新请求队列满了，插入下一个队列。
- 所有队列，轮流使用。

队列1 \*\* ← 当前服务队列  
队列2 \*\*\*\*\*  
队列3 \*\*\*\*\*  
队列4 \*\*\*\*\*  
队列5 \*\*\* ← 新请求队列  
队列6

- 单个队列服务期间，可以使用任意调度算法。如果是SCAN，这个算法就叫做 N-Step-SCAN。
- FCSCAN，N=2的特例，2个队列：磁臂单向移动，IO期间，新请求插入新队列。

# 优化旋转延迟 (硬件)



$$2T/N > \text{DMA传输的耗时}$$

|     |      |      |            |       |
|-----|------|------|------------|-------|
| ... | busy | done | Error Code | Error |
|-----|------|------|------------|-------|

状态寄存器 (端口)

命令寄存器 (端口)

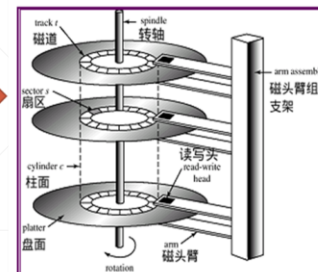
R/W GO

数据寄存器 (端口)

数据缓存

数据缓存存放

- 外设输入数据 (读操作)
- 计算机输出信息 (写操作)



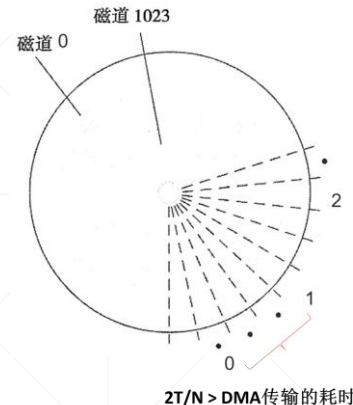
相邻扇区号对应的扇区，物理上一定不能相邻（DMA数据传输期间，磁头一定会**刚好**错过下个物理相邻的扇区首部）。  
 $\therefore$ 在同一磁道上，**n扇区和n+1扇区之间一定会相隔若干个扇区**。  
 下面的问题是：应该相隔多少个物理扇区为好呢？  
**答案是：**磁头越过这几个扇区需要的时间必须大于DMA芯片将512字节从磁盘数据缓存读入内存所需的时间。  
 精确的值需要根据硬件参数仔细计算，并留一定余量。



## 例

移动硬盘，技术参数如下：转速7500rpm，**平均寻道时间为4ms**，每磁道500扇区，同磁道内相邻逻辑扇区错开2个物理扇区。问：读取一个1.28M（十进制）字节的文件需要多长时间？

- 每分钟7500转，旋转1周时间为 8ms。
- 平均的旋转延迟 4ms
- 一个扇区的读取时间为 8ms/500
- 1.28M=2500 sector（扇区）



若**2500**扇区完全随机存放。

访问每个扇区都要经历寻道和旋转延时：

$$4\text{ms} + 4\text{ms} + (8\text{ms}/500)$$

顺序读取该文件需要：

$$2500 * (4\text{ms} + 4\text{ms} + (8\text{ms}/500)) = 20.04\text{s}$$

若**2500**扇区顺序排在相邻**5**个磁道内。假设，文件的**0#**数据块放在第一个磁道的**0#**扇区。所有磁道在同一个柱面。

访问第一个**500**个扇区：

$$\text{耗时} \approx 4\text{ms} + 4\text{ms} + 8\text{ms} * 3 = 32\text{ms}$$

后续**4**个磁道不需要寻道，每个磁道的访问时间：

$$\text{耗时} \approx 8\text{ms} + 8\text{ms} * 3 = 32\text{ms}$$

$$\text{顺序读取该文件需要耗时} \approx 32 + 32 * 4 = 160\text{ms}$$





# 考虑数据定位开销，优化文件访问速度

- 文件在磁盘上尽量连续存放。swap分区上的进程图像一定要连续存放。

# 预读技术

- 以同步方式读块设备时，进程不得不进入睡眠状态以等待数据传输结束，因此速度是相当低的。为了加快进程前进速度，提高中央处理机和块设备工作并行程度，最好在实际使用某字符块前，用异步方式提前将它读入缓存，在实际使用时，可立即从缓存中取用而无需等待。这种读字符块的方式称为预读。
- 按什么原则对磁盘数据块进行预读？用现在的行为推测未来要使用的数据块。
  - Unix V6，顺序读文件时进行预读：访问文件当前数据块时，顺带将下一个数据块读入缓存。
  - 现代操作系统，一次性把包含当前数据块的8个文件块读入缓存。
- 若文件系统将连续的大块文件数据存放在连续的磁盘扇区，预读技术将大幅减小磁臂移动的平均距离，提高IO请求队列的吞吐率。（Unix V6++文件系统不具备这个功能，现代文件系统全是这样的）



## 例：

- T时刻，进程PA执行如下操作顺序访问磁盘文件A。  
文件A总长2个数据块，0#、1#数据块分别存放在50#磁道，扇区号：n、n+2。

执行read系统调用读文件A 0~511字节，访问n#扇区  
.....处理读入的新数据（文件A的0~511字节）.....  
执行read系统调用读文件A 512~1024字节，访问n+2#扇区  
.....处理读入的新数据（文件A的512~1024字节）.....

并发的进程B执行如下操作访问存放在10000#磁道的另一个文件B，读写其中的某个数据块（扇区号m）。

执行read系统调用读文件B，访问 m#扇区

- 磁盘IO使用FCFS调度算法。假设，自由缓存队列有很多不脏的缓存块，n#、n+2#，m#扇区缓存不命中。磁头当前位于0#磁道。PA先执行。问（1）采用预读，服务这3个磁盘扇区，磁臂移动总距离



PA执行第1个read系统调用：将同步读请求n 和 异步读请求n+2送入IO请求队列后，入睡等待n#扇区读操作完成。  
PA放弃CPU后，PB执行，将IO请求m送入IO请求队列。

上图是这些操作完成后IO请求队列的样子。完成这3个请求，磁臂移动 $10000 - 50 = 9950$ 根磁道。

最后，我们看PA执行第2次read系统调用的细节：n#扇区IO完成后，第1个read系统调用返回。处理完新数据后，PA开始执行第2个read系统调用，它入睡等待n+2#扇区IO完成，磁盘中断处理程序解锁 n+2# 数据块。

## 例 (续) :

- T时刻, 进程PA执行如下操作顺序访问磁盘文件A。  
文件A总长2个数据块, 0#、1#数据块分别存放在50#磁道, 扇区号:  $n$ 、 $n+2$ 。

执行read系统调用读文件A 0~511字节, 访问 $n$ #扇区  
.....处理读入的新数据 (文件A 的0~511字节) .....  
执行read系统调用读文件A 512~1024字节, 访问 $n+2$ #扇区  
.....处理读入的新数据 (文件A 的512~1024字节) .....

并发的进程B执行如下操作访问存放在10000#磁道的另一个文件B, 读写其中的某个数据块 (扇区号 $m$ ) 。

执行read系统调用读文件B, 访问  $m$ #扇区

- 磁盘IO使用FCFS调度算法。假设, 自由缓存队列有很多不脏的缓存块,  $n$ #、 $n+2$ #,  $m$ #扇区缓存不命中。磁头当前位于0#磁道。PA先执行。问 (2) 不预读, 服务这3个磁盘扇区, 磁臂移动总距离



PA执行第1个read系统调用: 将同步读请求 $n$  送IO请求队列, 入睡等待 $n$ #扇区读操作完成。

PA放弃CPU后, PB执行, 将IO请求 $m$ 送入IO请求队列。PA执行第2个read系统调用: 将同步读请求 $n+2$  送IO请求队列

上图是这些操作完成后IO请求队列的样子。完成这3个请求, 磁臂移动  $2 * (10000 - 50)$  根磁道。

相邻扇区IO请求没法挨在一起, 整个磁臂移动轨迹有很多折返, 磁盘吞吐率下降了。



## 附加题：

- T时刻，进程PA执行如下操作顺序访问磁盘文件A。  
文件A总长2个数据块，0#、1#数据块分别存放在50#磁道，扇区号：n、n+2。  
执行read系统调用读文件A 0~511字节，访问n#扇区  
.....处理读入的新数据（文件A的0~511字节）.....  
执行read系统调用读文件A 512~1024字节，访问n+2#扇区  
.....处理读入的新数据（文件A的512~1024字节）.....

并发的进程B执行如下操作访问存放在10000#磁道的另一个文件B，读写其中的某个数据块（扇区号m）。  
执行read系统调用读文件B，访问m#扇区

- **磁盘IO使用电梯调度算法。**当前磁头向大号磁道移动。假设，自由缓存队列有很多不脏的缓存块，n#、n+2#，m#扇区缓存不命中。磁头当前位于0#磁道。PA先执行。

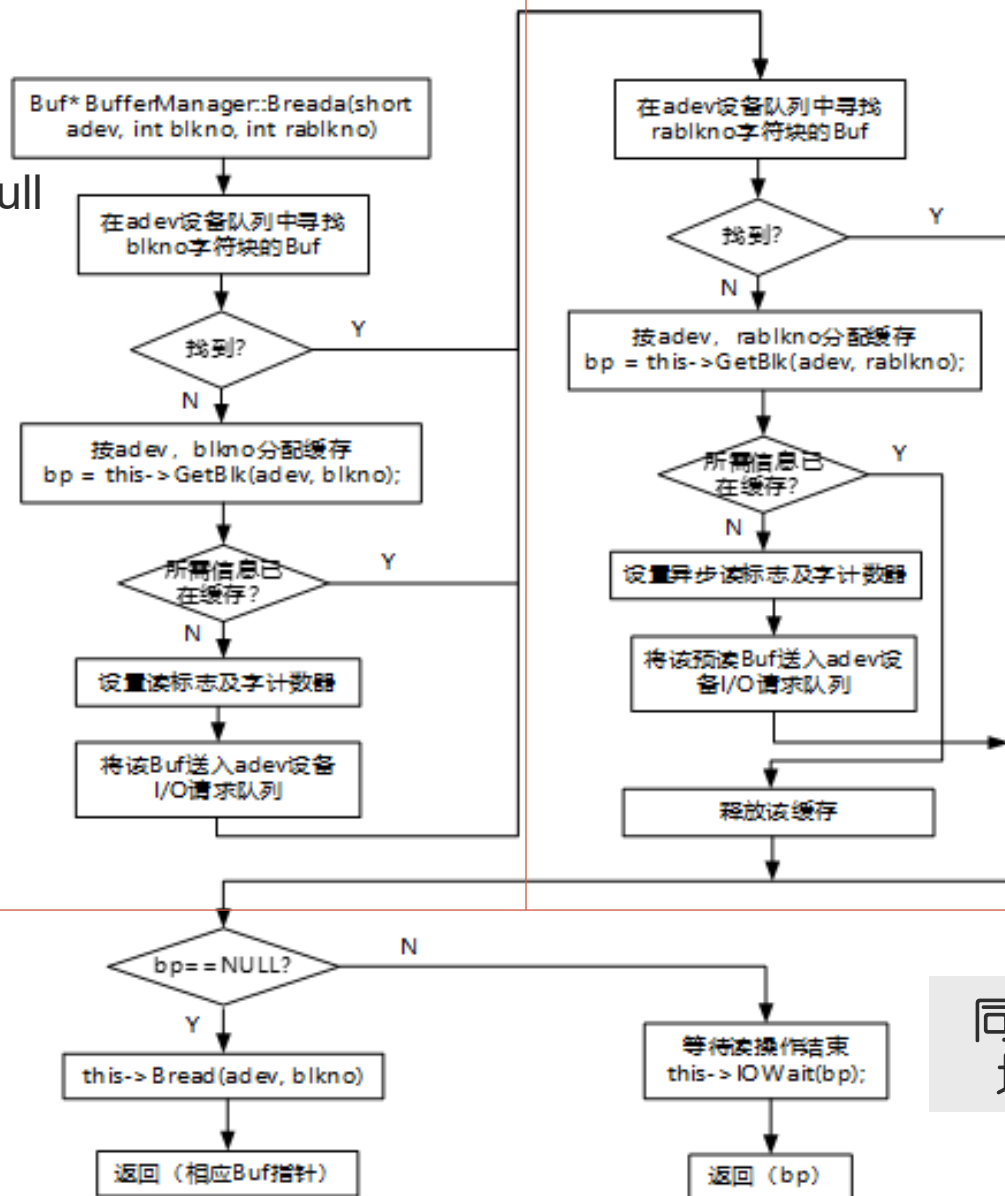
问（1）不预读，服务这3个磁盘扇区，磁臂移动总距离（2）预读，服务这3个磁盘扇区，磁臂移动总距离  
（3）预读技术能否进一步提升电梯调度算法的性能，缩短磁臂移动总距离？

代码:

bp=null

读入当前块 n

读入预读块 n+2



同步等待当前块缓存就绪



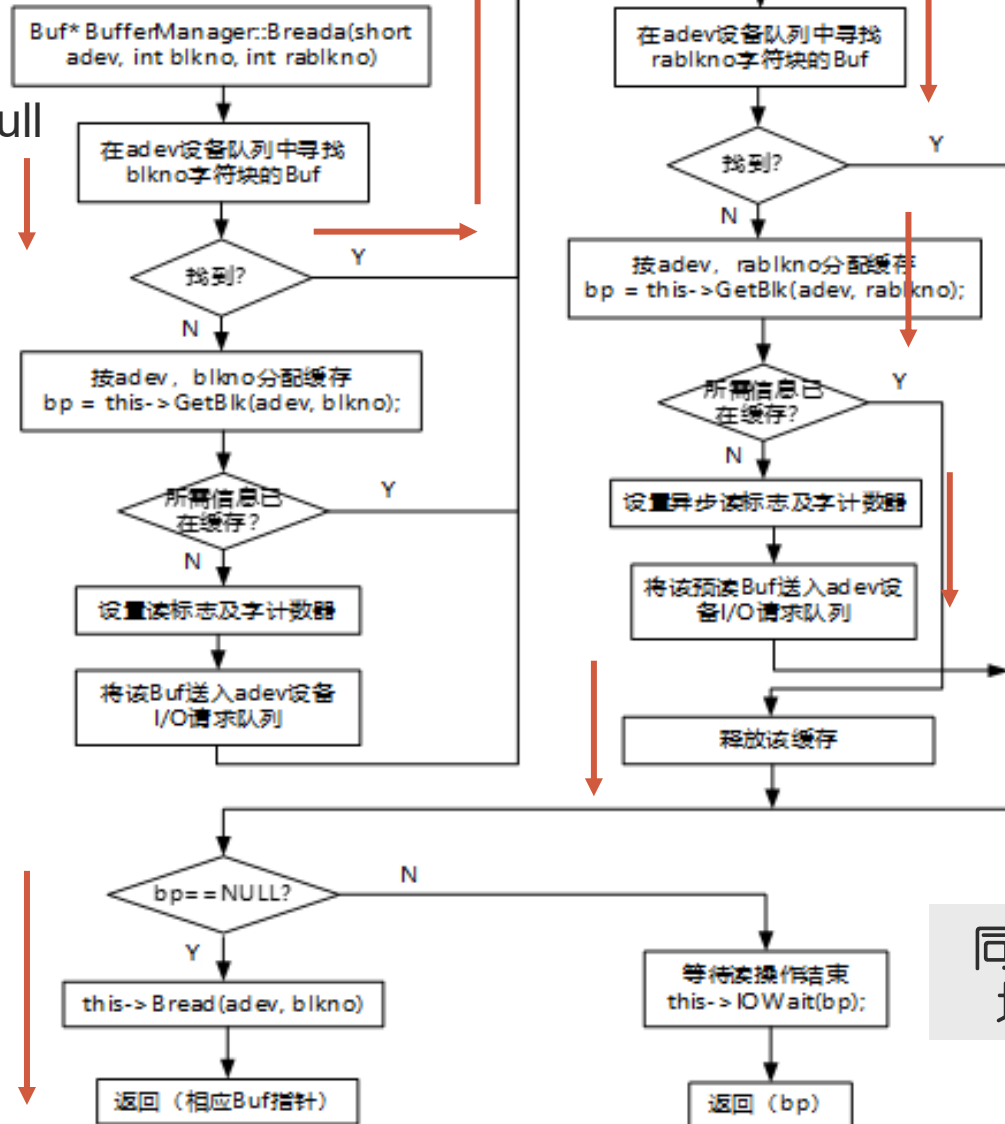
# n、n+2缓存命中

读入当前块 n

读入预读块 n+2

同步等待当前块缓存就绪

bp=null





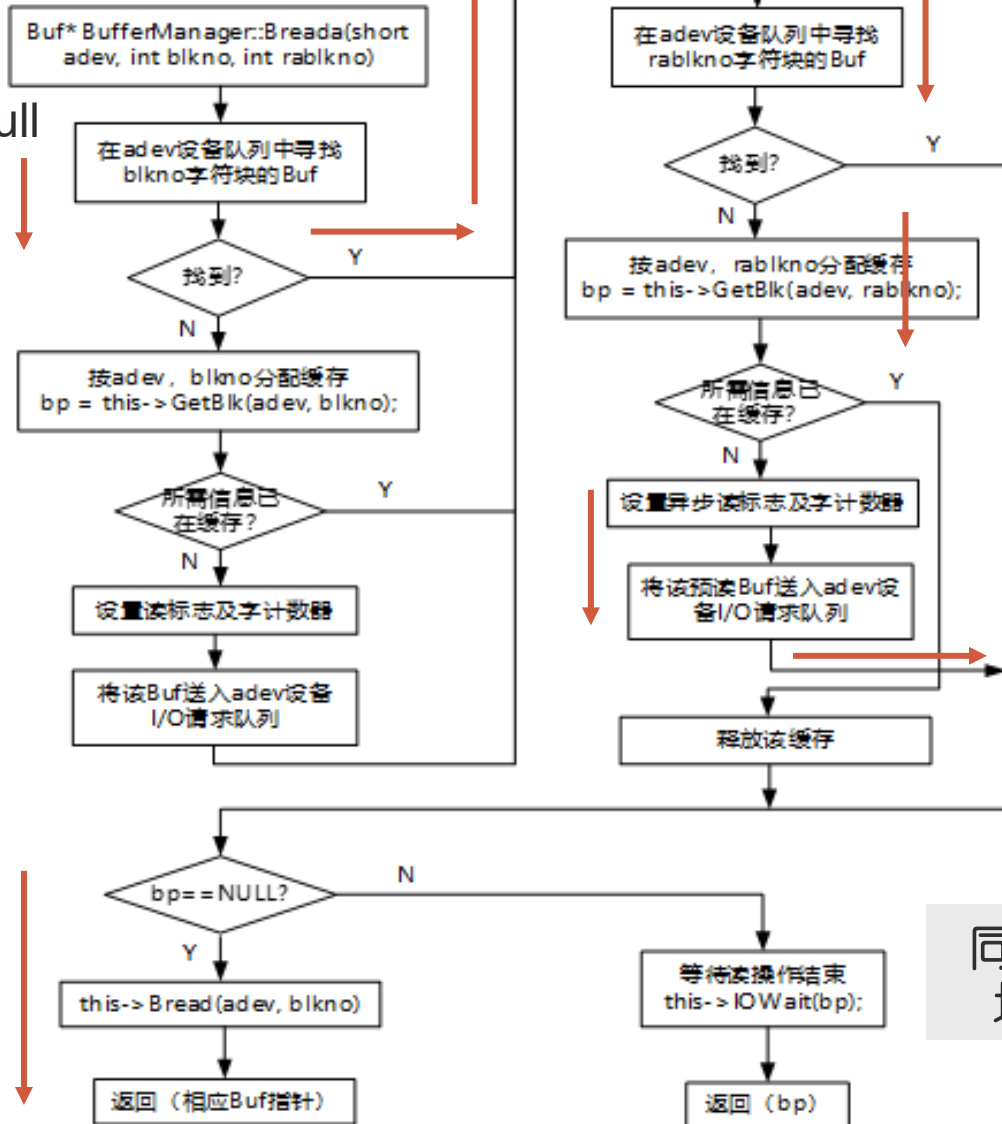
# n 缓存命中 n+2缓存不命中

读入当前块 n

读入预读块 n+2

同步等待当前块缓存就绪

bp=null



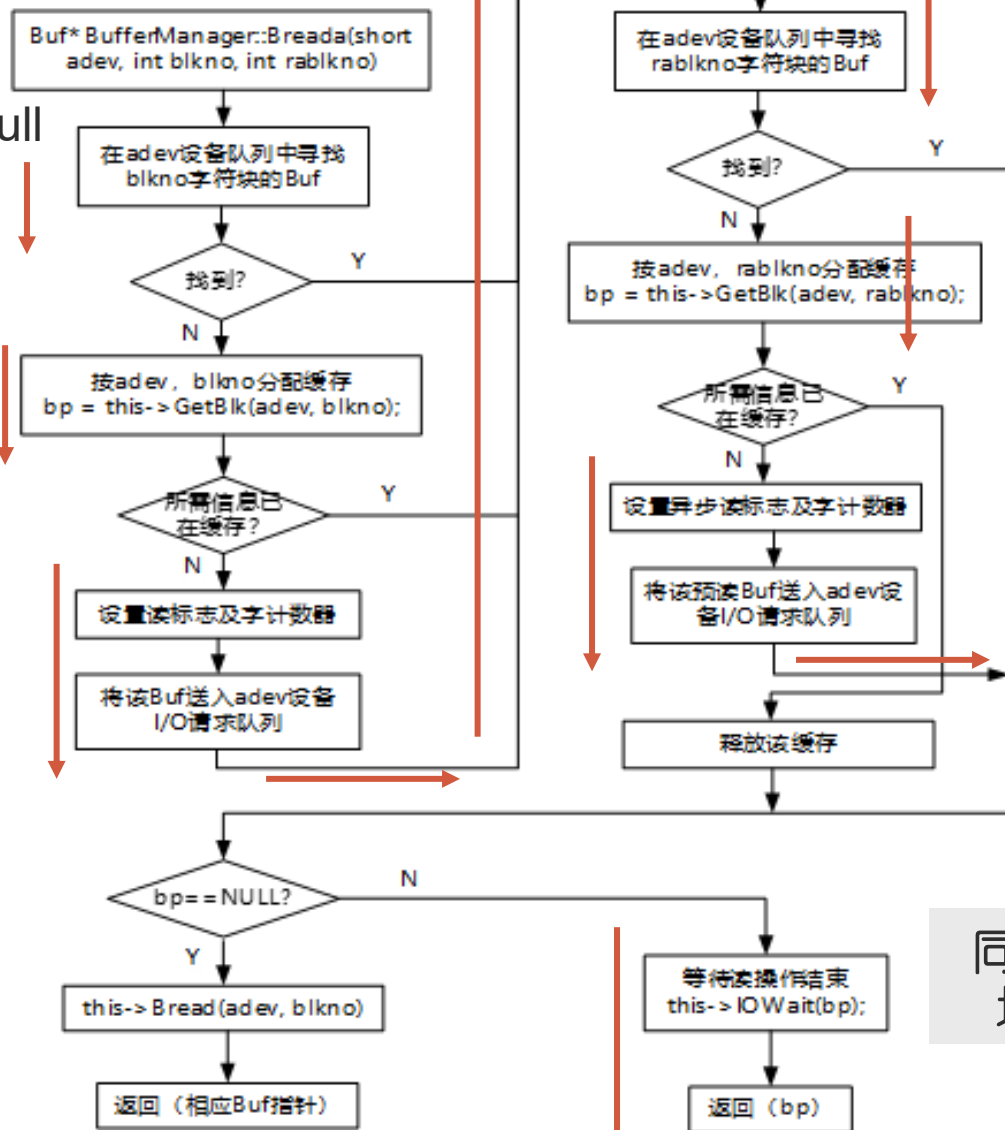
# n 缓存不命中 n+2缓存不命中



读入当前块 n

读入预读块 n+2

bp=null



同步等待当前块缓存就绪